



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

SISTEMAS DISTRIBUIDOS

Laboratorio S11

Mexi Banco: dos bases más y una caché (persistencia políglota)

Nombre del alumno:	Yukioayax Canek Gabriel Hernández
Profesor:	Oscar Manuel Ruiz Hurtado
Grupo:	2
Semestre en curso:	2027-1

Ciudad Universitaria, CDMX

Índice

1. Introducción **2**

2. Parte 0: el punto de partida **2**

3. Parte 1: la caché del saldo en Redis **3**
3.1. El código 4
3.2. Las tres copias comparten la caché 4
3.3. La copia se borra al transferir 5
3.4. Mis medidas 6

4. Parte 2: los avisos en MongoDB **7**
4.1. El código 7
4.2. Las tres formas 8
4.3. Los últimos 10 10
4.4. El índice y los dos explain 10

5. Parte 3: el evento duplicado **12**

6. Parte 4: el dato viejo **12**

7. Reto: Redis se cae **13**
7.1. Sin protección 14
7.2. Dónde atrapar el error 14
7.3. La primera versión creó dinero 14
7.4. La versión final 16

8. Preguntas de cierre **18**

9. Conclusiones **19**

1 Introducción

En este laboratorio Mexi Banco pasó de usar solo PostgreSQL a usar tres almacenes, y cada uno guarda lo que mejor sabe guardar, Postgres sigue siendo la verdad del dinero (cuentas, movimientos y transferencias), **Redis** guarda una copia del saldo solo para mostrarlo rápido, y **MongoDB** guarda la bitácora de avisos, que ahora tiene una forma distinta según el canal (SMS, correo o push). La idea de fondo es separar qué dato es la verdad y qué dato es derivado, lo derivado se puede reconstruir y por eso se vale tenerlo en otro motor y hasta un poco atrasado, pero nunca se usa para decidir.
Partí de la rama `06-poliglota-base` y lo corrí todo en Docker en mi máquina. Como el puerto 15672 lo ocupa el RabbitMQ de mi proyecto, levanté el panel en el 15673 (`PUERTO_RABBIT_PANEL=15673`, como dice el compose), y no cambié nada más del `docker-compose.yml`. Los comandos los corrí en Git Bash, y los que usé varias veces los dejé en `codigo/scripts/`.

2 Parte 0: el punto de partida

Primero levanté todo sin tocar código, los trece contenedores salieron *healthy*, Redis contestó `PONG` y en Mongo había 0 avisos.

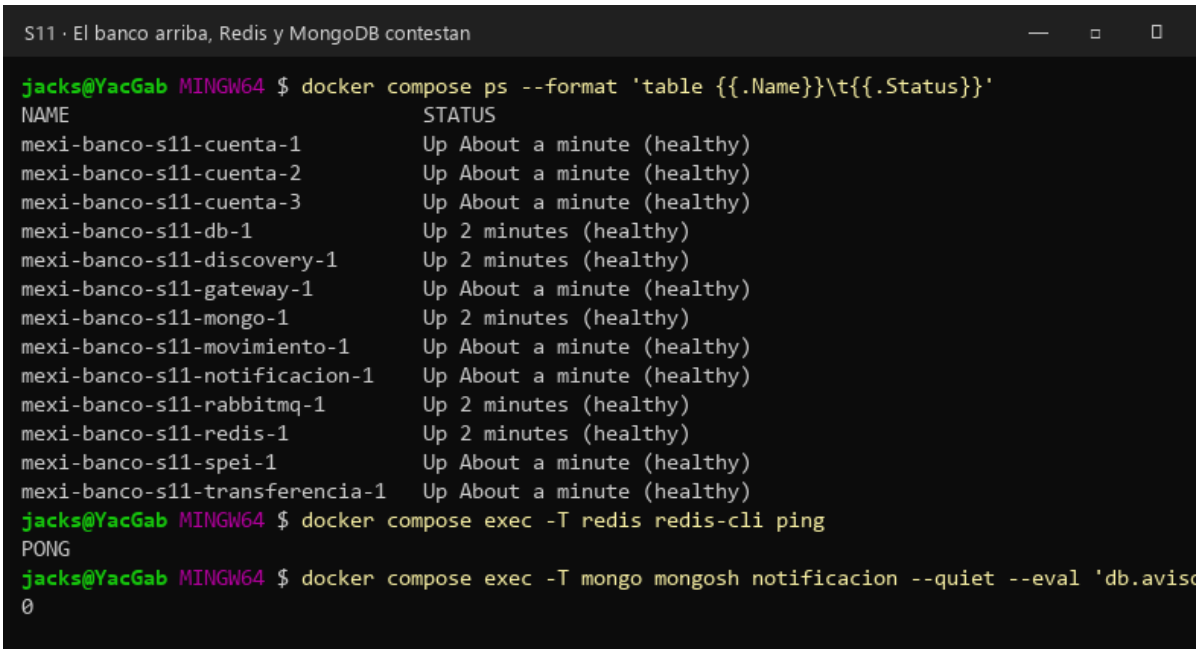


Figura 1: Los trece contenedores sanos, Redis contesta `PONG` y la colección `avisos` está vacía.

Abrí las cuentas de Ana (1000), Beto (500) y Carla (300), las tres con 201, e hice la transferencia de 200 de Ana a Beto, que salió 201 con estado `COMPLETADA`.

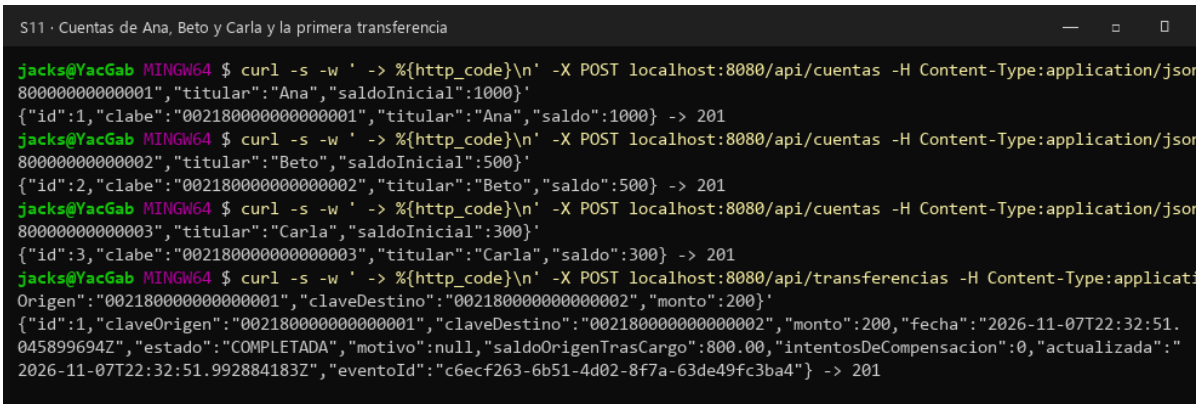


Figura 2: Alta de Ana, Beto y Carla, y la primera transferencia de 200.

Después pedí el saldo de Ana diez veces, en las diez respuestas el `leidaEn` es distinto, o sea que cada una se leyó de Postgres en ese momento, el log de las tres copias de `cuenta` tiene **12** líneas de `leida de Postgres`, las 10 consultas más 2 lecturas que hizo la transferencia, en Redis `keyspace_hits:0` y `keyspace_misses:0`, porque nadie lo usa todavía, y en Mongo sigue habiendo 0 avisos, los avisos de la S09 siguen en la bandeja de Postgres (`GET /api/notificaciones?clabe=...`), donde está el de Beto por los 200.

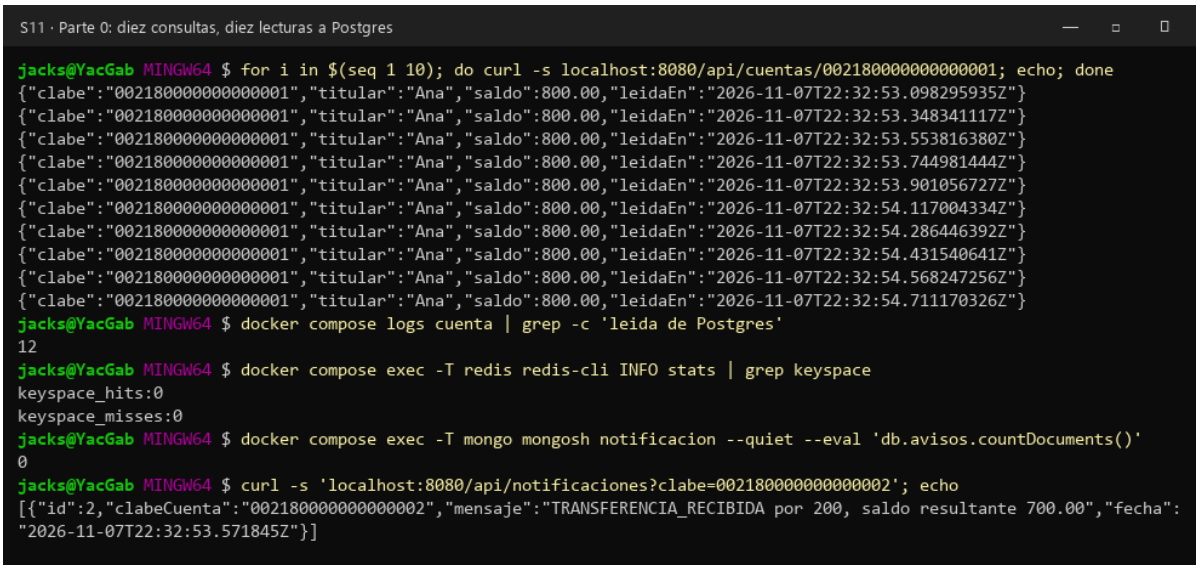


Figura 3: Diez consultas, diez lecturas a Postgres: el “antes”.

O sea, en el punto de partida cada vez que alguien ve su saldo es una lectura a Postgres, aunque el saldo no haya cambiado.

3 Parte 1: la caché del saldo en Redis

3.1 El código

Son tres archivos de `cuenta`. En `ConfiguracionCache` puse `@EnableCaching`, sin eso las anotaciones de caché se ignoran sin avisar, y un `@Bean` que le cambia tres cosas a la caché `cuentas`:

```
RedisCacheConfiguration cuentas = RedisCacheConfiguration.defaultCacheConfig()
    .entryTtl(ttl) // cada copia vive 30 s y luego Redis la borra sola
    .disableCachingNullValues()
    .serializeValuesWith(SerializationPair.fromSerializer(
        new Jackson2JsonRedisSerializer<>(json, CuentaVista.class))); // JSON legible
return builder -> builder.withCacheConfiguration(CUENTAS, cuentas)
    .transactionAware(); // el DEL espera al commit
```

El TTL es para que una copia vieja no se quede para siempre si alguien cambia el saldo por fuera, el serializador JSON es para que en Redis se pueda leer el valor con `redis-cli GET` y no salga en binario de Java, y `transactionAware()` hace que el borrado espere a que la transacción del cargo o del abono haga commit.
En `ConsultaDeSaldo.consultar`, que es lo que contesta `GET /cuentas/{clabe}`, puse `@Cacheable(cacheNames = CUENTAS, key = "#clabe")`, es cache-aside, si la copia está en Redis contesta Redis y el método ni corre, y si no está, corre, lee Postgres y Spring guarda el resultado. Y en `CuentaService.cargar` y `abonar` puse `@CacheEvict` con la misma llave, cuando el saldo cambia la copia se **borra**, no se actualiza.
Lo que no toqué es `CuentaService.consultar`, que es la lectura con la que `cargar` decide si alcanza el saldo, esa siempre va a Postgres, por eso la consulta que muestra vive en otra clase, y además si pusiera `@Cacheable` en un método que se llama desde su misma clase, la llamada no pasaría por el proxy de Spring y la anotación no haría nada.

3.2 Las tres copias comparten la caché

Antes de correrlo mi predicción era que, aunque contestaran copias distintas de `cuenta`, Postgres se iba a leer una sola vez, porque Redis es uno solo para las tres. Y así fue, en las seis consultas el `X-Instancia` va cambiando entre las tres copias (`ccd15351c4bc`, `0cc664fdeea3` y `ab911463d4bf`), pero el `leidaEn` es exactamente el mismo en las seis, y en los últimos 30 segundos hubo **una** lectura a Postgres, en Redis está el JSON de `CuentaVista` con el saldo de 800 y su TTL era de 25 segundos, si la caché fuera local, dentro de cada copia, se hubiera leído Postgres tres veces.

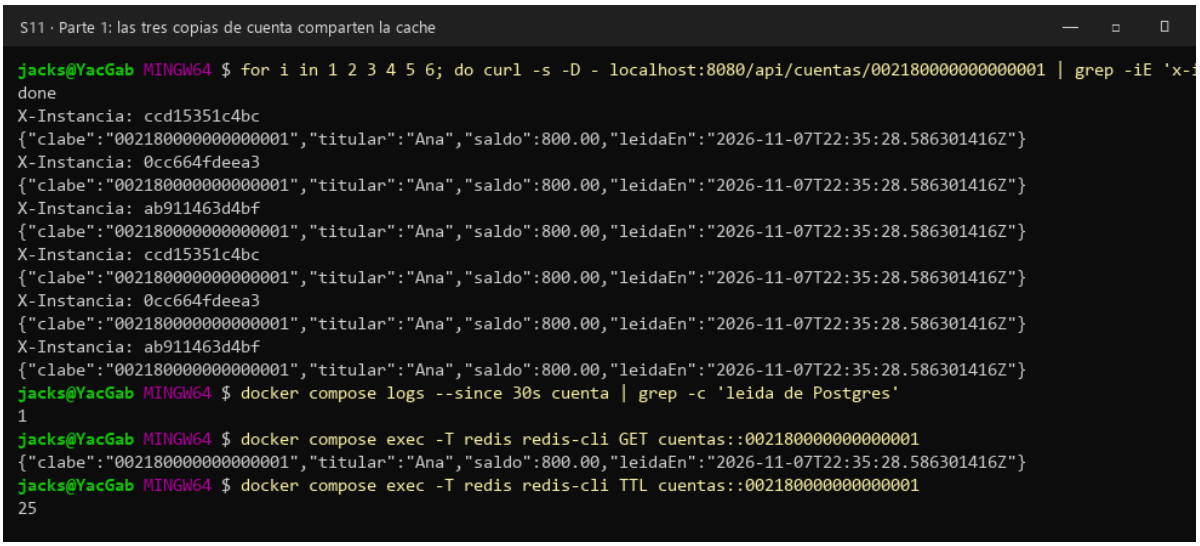


Figura 4: Seis consultas contestadas por tres copias distintas con el mismo `leidaEn`, una lectura a Postgres, el JSON en Redis y su TTL.

3.3 La copia se borra al transferir

Dejé corriendo `redis-cli MONITOR` en otra terminal, transferí 10 de Ana a Beto, y justo después `EXISTS cuentas::002180000000000001` dio **0**, la copia ya no estaba, y el siguiente GET trajo el saldo nuevo, 790, con un `leidaEn` nuevo. En el MONITOR se ve el orden: los `GET` de Ana y de Beto (la saga consulta las cuentas por el mismo endpoint, y como Beto no estaba hizo un `SET`), luego los dos `DEL`, de Ana y de Beto, después del commit del cargo y del abono, y al final mi consulta, que fue un miss, leyó Postgres y dejó la copia nueva con `SET ... PX 30000`.

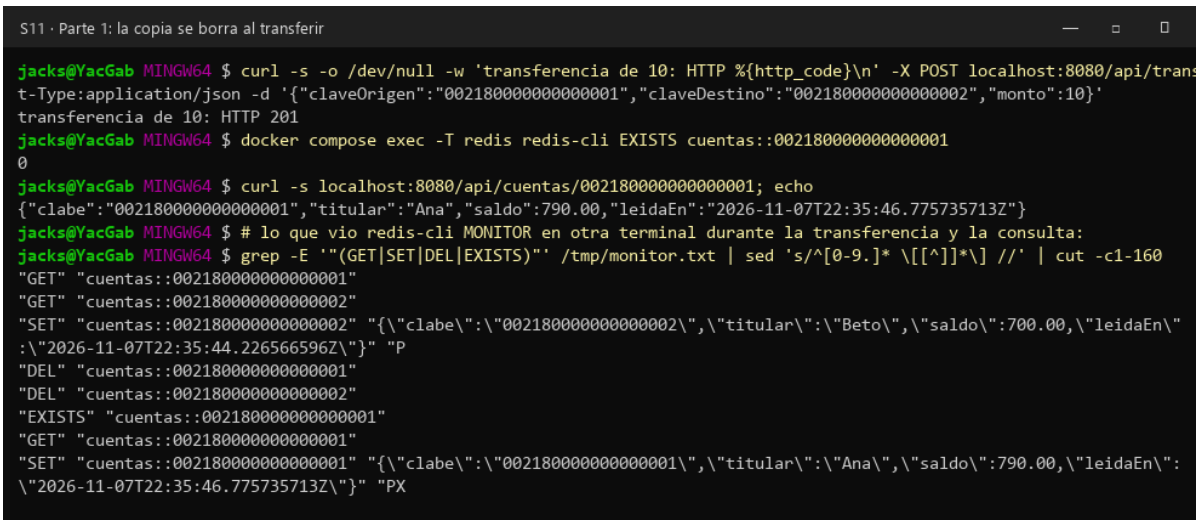


Figura 5: La transferencia de 10, `EXISTS` en 0, el saldo nuevo y lo que vio el MONITOR de Redis.

3.4 Mis medidas

Hit ratio. Puse los contadores en cero con `CONFIG RESETSTAT` y medí los dos escenarios:

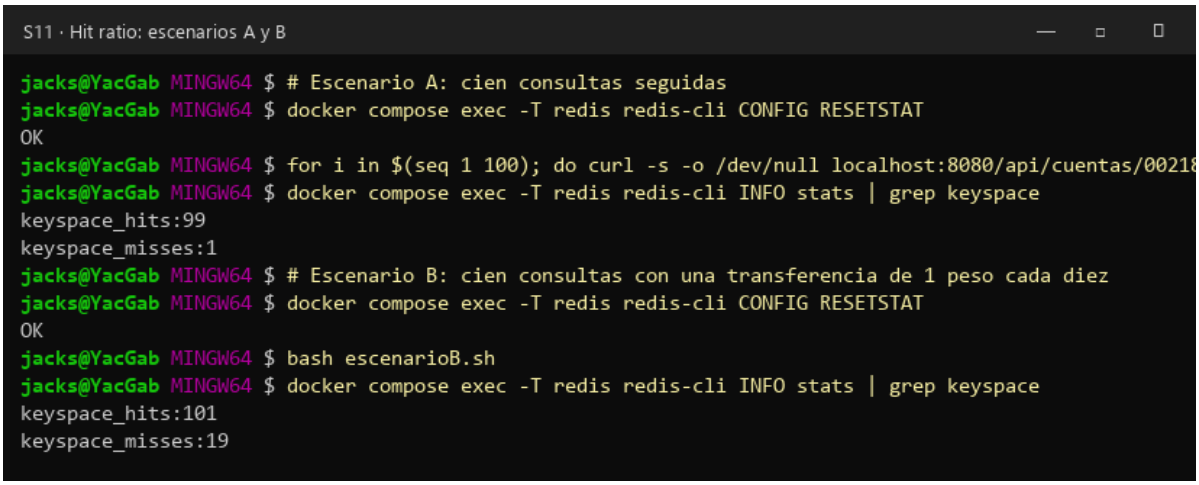


Figura 6: Escenario A: 99 hits y 1 miss. Escenario B: 101 hits y 19 misses.

Escenario	hits	misses	hit ratio
A: 100 consultas seguidas	99	1	99.0 %
B: 100 consultas, una transferencia de 1 cada diez	101	19	84.2 %

En A el único miss es la primera consulta, la copia anterior ya había caducado, y las otras 99 salen de Redis. En B baja porque cada transferencia borra la copia de Ana y la de Beto, entonces la siguiente consulta de Ana falla (10 misses) y la saga, al revisar a Beto, también falla (9 más), además las transferencias hacen sus propios `GET` a Redis, por eso en B hay 120 lecturas y no 100.

Latencia. Hice tres corridas de 100 consultas con caché y tres con `CACHE_TIPO=none`, que apaga la caché sin tocar código, y en cada una conté las lecturas a Postgres (`codigo/scripts/latencia.sh`). Mi predicción era que con caché iba a ser bastante más rápido, y no fue así:

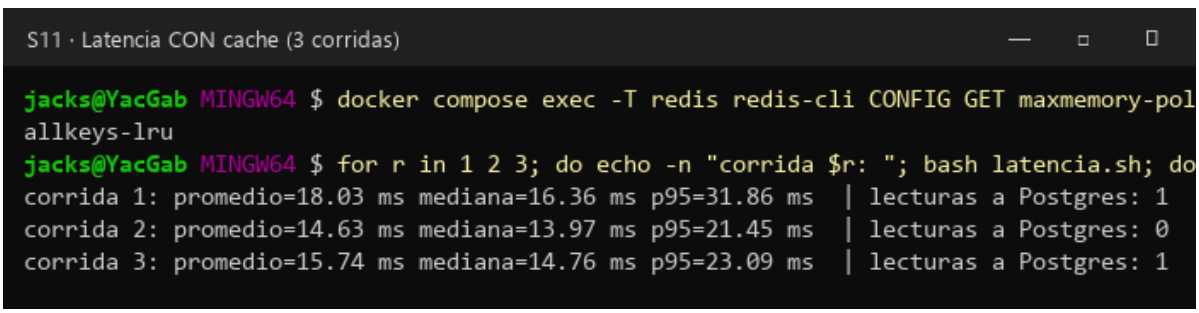


Figura 7: Latencia con caché, tres corridas.

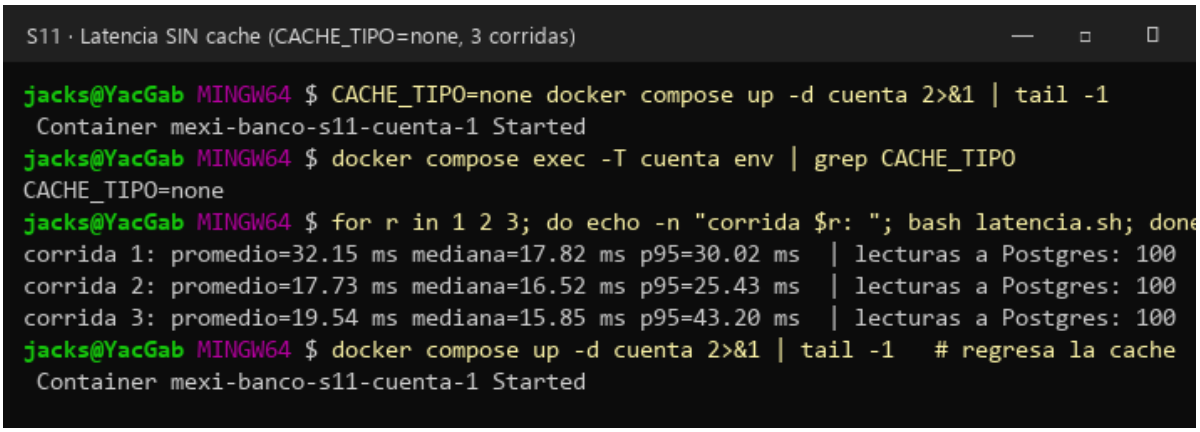


Figura 8: Latencia sin caché (`CACHE_TIPO=none`), tres corridas.

	Corrida	promedio (ms)	mediana (ms)	p95 (ms)	lecturas a Postgres
Con caché	1	18.03	16.36	31.86	1
	2	14.63	13.97	21.45	0
	3	15.74	14.76	23.09	1
Sin caché	1	32.15	17.82	30.02	100
	2	17.73	16.52	25.43	100
	3	19.54	15.85	43.20	100

La mediana con caché quedó entre 14 y 16 ms y sin caché entre 16 y 18 ms, apenas uno o dos milisegundos, porque en mi laptop Postgres tiene la tabla en memoria y está a un salto de red, igual que Redis, y casi todo el tiempo se va en el gateway, el balanceo y HTTP, no en la lectura, la diferencia grande está en las lecturas a Postgres, con caché fueron 0 o 1 por cada 100 consultas, y sin caché fueron 100 de 100.

4 Parte 2: los avisos en MongoDB

4.1 El código

En `BitacoraDeAvisos` llené los cuatro métodos. `documento(...)` arma cada aviso como un `org.bson.Document`, primero los campos que tienen todos y luego un `switch` sobre el `Contacto` que agrega los del canal, como `Contacto` es una interfaz sellada, si me falta un canal no compila:

```
Document aviso = new Document("_id", evento.eventoId() + ":" + destinatario.clabe())
    .append("eventoId", evento.eventoId().toString())
    .append("transferenciaId", evento.transferenciaId())
    .append("clabe", destinatario.clabe())
    .append("tipo", destinatario.tipo())
    .append("monto", new Decimal128(evento.monto())) // dinero: Decimal128, nunca double
    .append("saldo", new Decimal128(destinatario.saldo()))
    .append("fecha", Date.from(evento.fecha())); // la fecha del evento, no la de hoy
switch (contacto) {
```

```
    case Contacto.Sms sms -> aviso.append("canal", "SMS").append("telefono", sms.
        telefono()).append("texto", texto);
    case Contacto.Correo c -> aviso.append("canal", "CORREO").append("correo", c.correo
        ())
                                .append("asunto", asunto(destinatario.tipo())).append("
                                    cuerpo", texto);
    case Contacto.Push p -> aviso.append("canal", "PUSH").append("token", p.token())
                               .append("titulo", asunto(destinatario.tipo())).append("
                                   cuerpo", texto);
}
```

`registrar(evento)` inserta los avisos uno por uno y cuenta los nuevos, si Mongo contesta `DuplicateKeyException` es que ese `_id` ya estaba, no es error, no se cuenta y sigue con el siguiente, va uno por uno porque un insert de varios se detiene en el primer repetido. `crearIndice()` crea `clabe_fecha` con `{clabe: 1, fecha: -1}` al arrancar, y `ultimos(clabe)` es una `Query` con `clabe` igual, ordenada por `fecha` descendente y con `limit(10)`. En el consumidor cambié la línea que guardaba en la bandeja de Postgres por `nuevos = bitacora.registrar(evento)`, solo en Mongo, porque escribir en los dos sería una doble escritura sin transacción común, el mismo problema de la S09.

4.2 Las tres formas

Al arrancar, el log dice `[avisos] indice clabe_fecha listo en avisos`. Hice una transferencia de 50 de Ana a Beto y otra de 50 de Ana a Carla, y en Mongo quedaron cuatro avisos con tres formas distintas, los dos de Ana son `SMS` con `telefono` y `texto`, el de Beto es `CORREO` con `correo`, `asunto` y `cuerpo`, y el de Carla es `PUSH` con `token`, `titulo` y `cuerpo`, todos con `monto` y `saldo` en `Decimal128` y la `fecha` del evento.

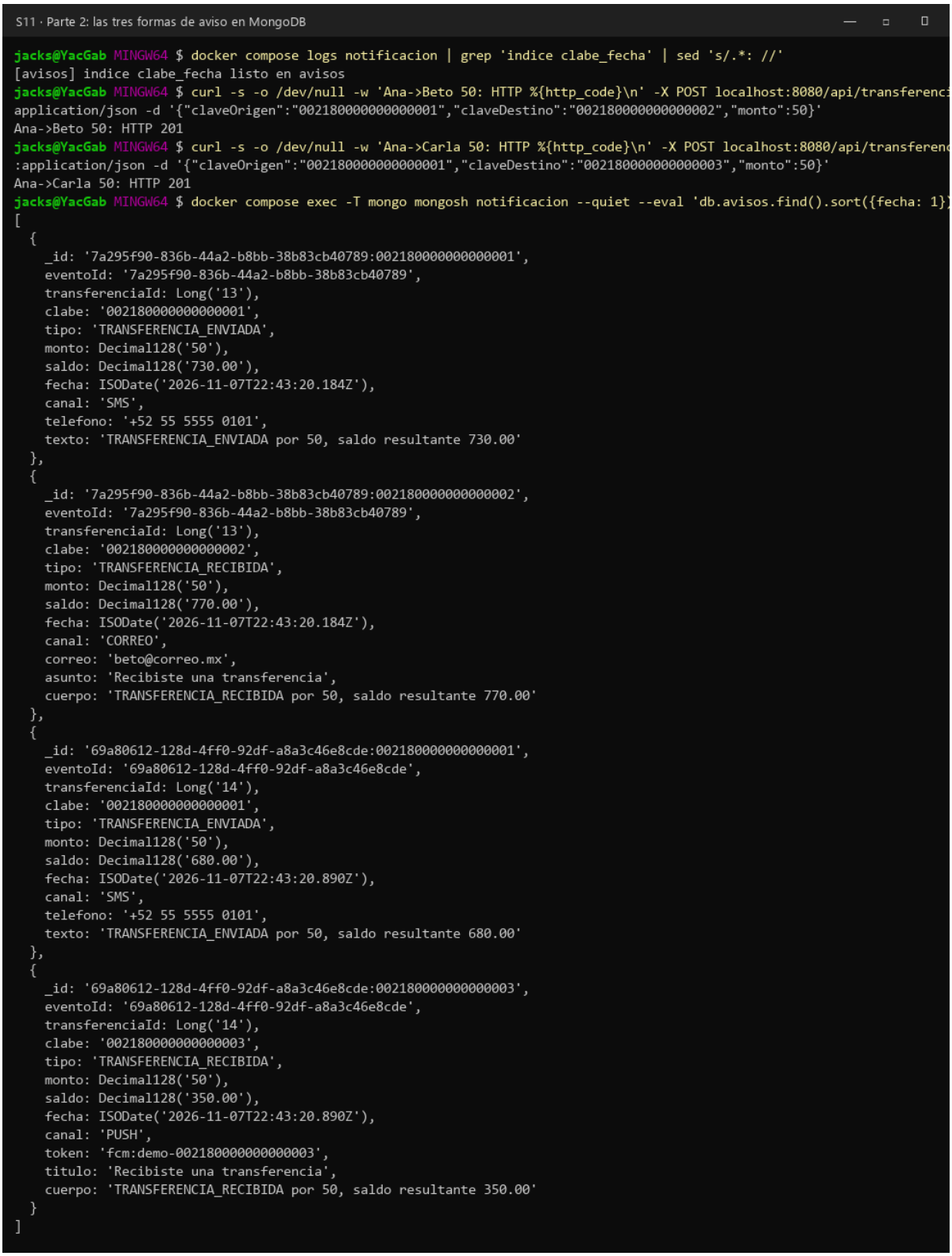


Figura 9: Los cuatro avisos en `mongosh`: dos SMS de Ana, un CORREO de Beto y un PUSH de Carla.

Y si pido solo los que tienen `telefono`, salen nada más los dos de Ana, nadie hizo un `ALTER TABLE` para que existiera ese campo, el esquema lo aplica quien lee.

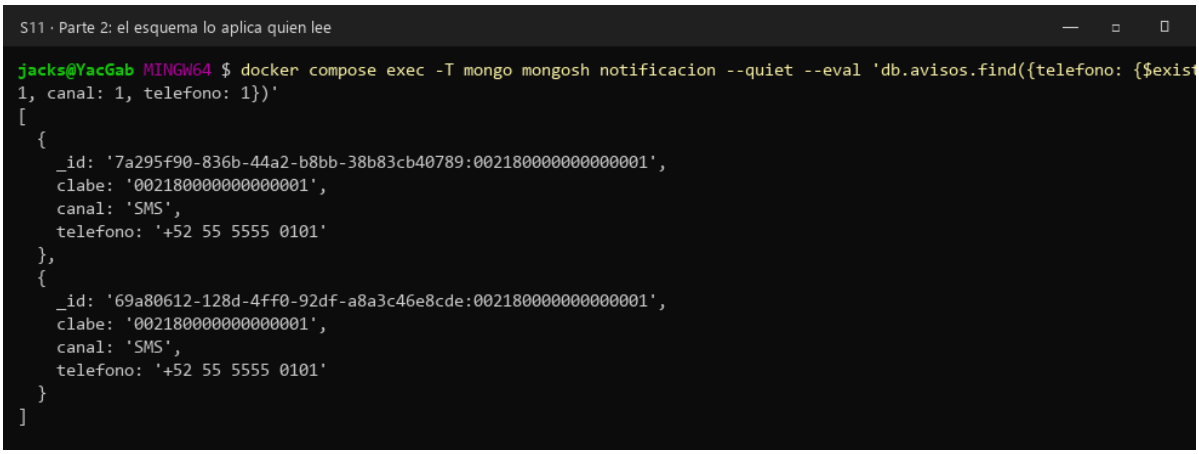


Figura 10: Solo los avisos con `telefono`: los dos SMS de Ana.

4.3 Los últimos 10

Hice doce transferencias de 1 peso de Ana a Beto, Beto ya tiene 13 avisos en Mongo, pero el endpoint `GET /api/notificaciones/{clabe}` regresa exactamente **10**, todos `CORREO`, del más reciente al más viejo.

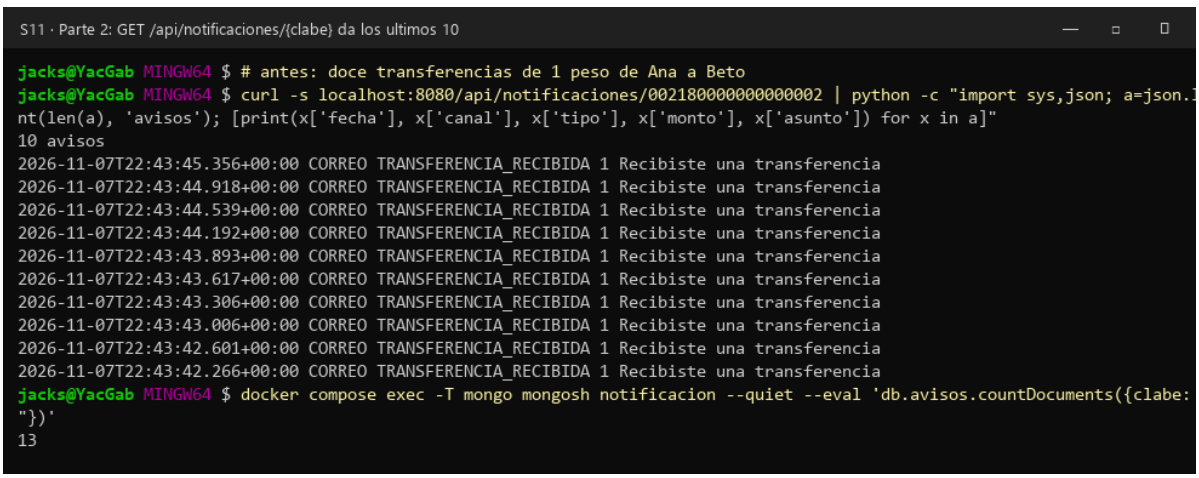


Figura 11: El endpoint da 10 avisos de Beto, del más nuevo al más viejo, aunque en Mongo tiene 13.

4.4 El índice y los dos explain

`getIndexes()` muestra `_id_` y `clabe_fecha`. Para comparar los dos planes hice un script (`explain.js`) que corre la misma consulta de `ultimos()` con y sin índice (con `.hint({$natural: 1})`) y resume el `explain(.executionStats")`, y también corrí los dos `explain` de la guía filtrando las líneas que importan:

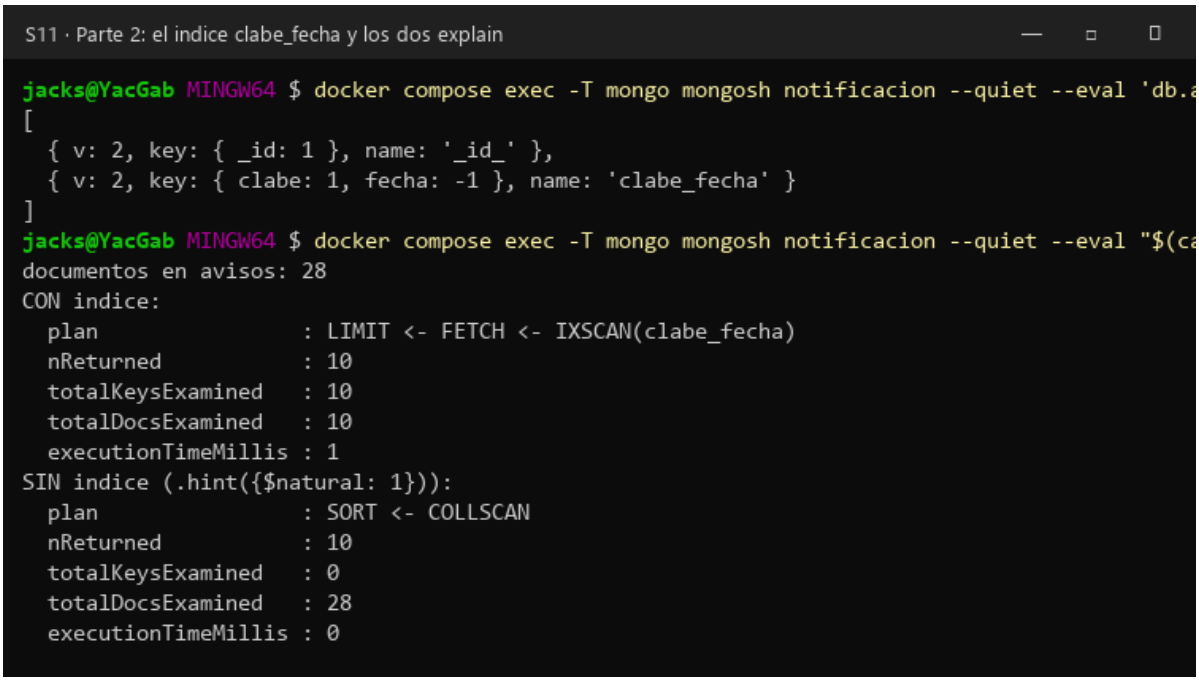


Figura 12: Los índices de `avisos` y el resumen de los dos planes, con 28 avisos en la colección.

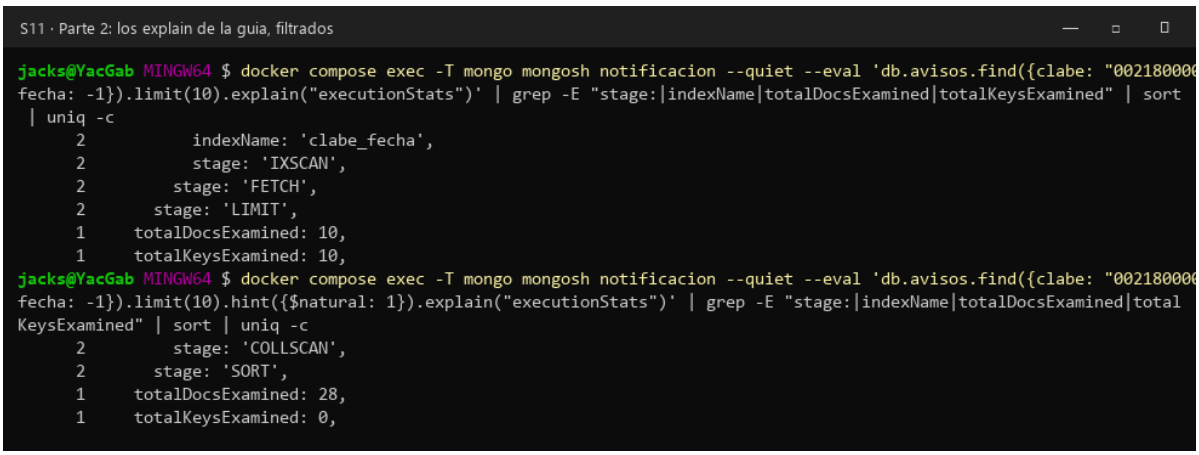


Figura 13: Los dos `explain` de la guía, filtrados: etapas y documentos examinados.

	Etapas	Documentos examinados
Con índice	`LIMIT <- FETCH <- IXSCAN(clabe_fecha)`	10 (y 10 llaves)
Sin índice	`SORT <- COLLSCAN`	28, todos

Con el índice Mongo va directo a las llaves de Beto, ya ordenadas por fecha, y lee solo los 10 que regresa, sin el índice recorre los 28 documentos de la colección y luego los ordena en memoria, con 28 la diferencia en tiempo es cero milisegundos, pero el número que crece es el de documentos examinados, con millones de avisos sería leer 10 contra leer millones.

5 Parte 3: el evento duplicado

Publiqué dos veces el mismo evento por la API del panel de RabbitMQ (`codigo/scripts/publicar_evento.sh`), con el `eventoId` fijo `11111111-2222-4333-8444-555555555555` y CLABEs inventadas (`...091` y `...092`). Mi predicción era que iban a quedar 2 documentos y no 4, porque el segundo evento arma los mismos dos `_id`. Así fue, las dos publicaciones dijeron `routed: true`, `countDocuments` da **2**, y en el log la primera entrega dice `2 aviso(s) nuevo(s)`, y la segunda tiene las dos líneas de `ya estaba: evento repetido, no se duplica` y termina con `0 aviso(s) nuevo(s)`.

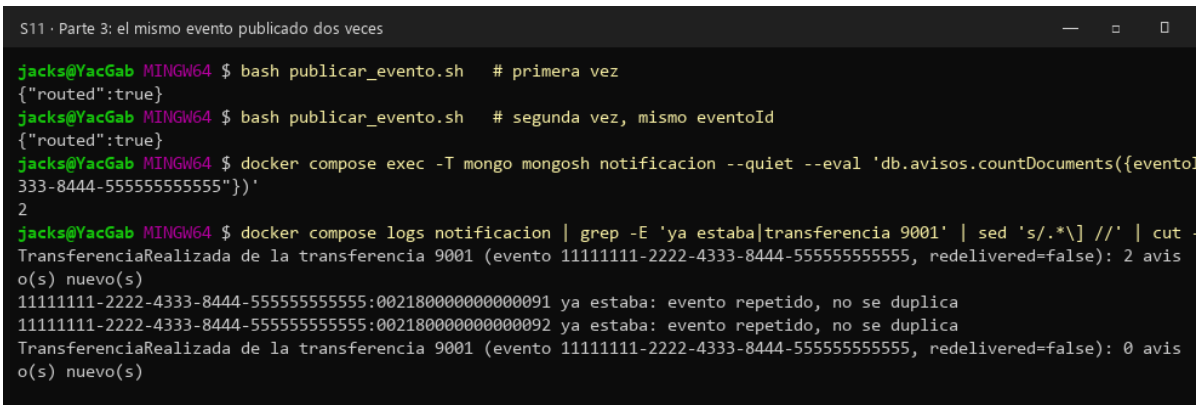


Figura 14: El mismo evento dos veces deja dos avisos, y el log dice que el segundo intento ya estaba.

Es la misma idempotencia que hice en la S09 con la tabla `evento_procesado`, pero aquí no hace falta tabla aparte, el `_id` compuesto `eventoId:clabe` es la llave única.

6 Parte 4: el dato viejo

Consulté a Ana, su saldo era 668 y quedó en la caché, y luego le cambié el saldo directo en Postgres a 5, como haría un script mal hecho, eso no pasa por el `@CacheEvict`, así que nadie borró la copia. Después pedí el saldo cada 5 segundos mostrando el TTL, y mientras tanto, en segundo plano, mandé una transferencia de 100 de Ana a Beto.
Mi predicción para la transferencia era que no iba a pasar, porque el cargo se autoriza con `CuentaService.consultar`, que lee Postgres, y ahí ya hay 5.

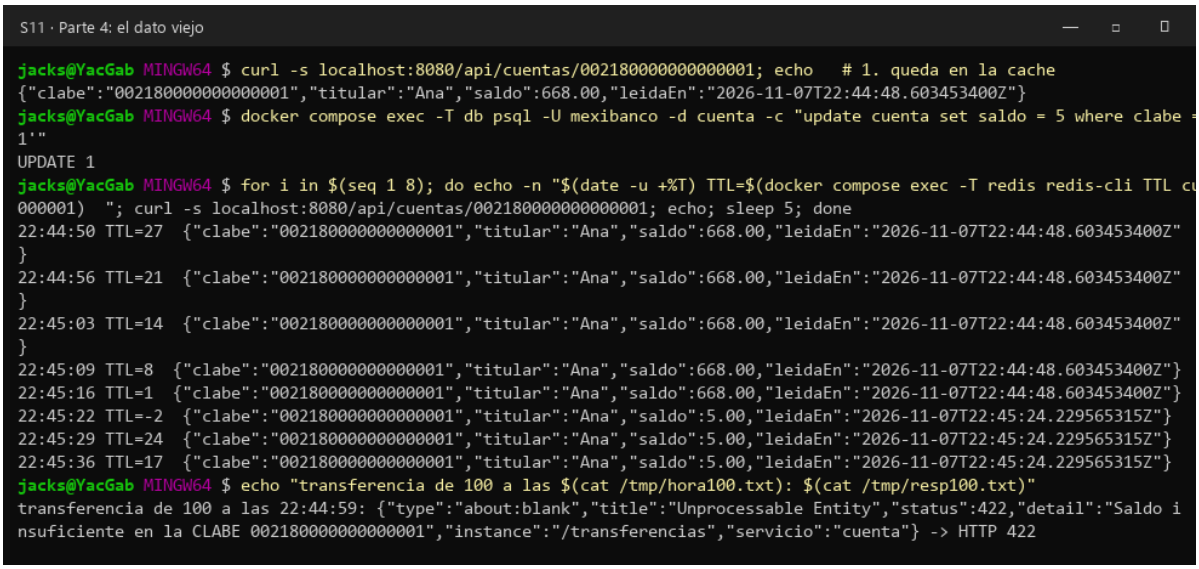


Figura 15: El cliente sigue viendo 668 mientras el TTL baja, la transferencia de 100 da 422 y al caducar la copia aparece el 5.

Mientras el TTL bajó de 27 a 1, el cliente siguió viendo **668** con el mismo `leidaEn` de las 22:44:48, y en cuanto caducó (TTL -2) el GET leyó Postgres y por fin mostró **5**, con un `leidaEn` nuevo de las 22:45:24, o sea que el cliente vio un saldo que ya no existía durante unos **35 segundos**, desde que cambié el saldo hasta la nueva lectura, se pasa un poco de los 30 del TTL porque la copia ya llevaba un par de segundos cuando hice el `UPDATE` y la siguiente consulta fue en la vuelta de 5 segundos. El `leidaEn` lo deja ver muy claro, dice de cuándo es el dato que estás viendo, y durante todo ese rato decía 22:44:48.
La transferencia de 100, que mandé a las 22:44:59 con la pantalla todavía diciendo 668, contestó **422**, `Saldo insuficiente en la CLABE 002180000000000001`, y lo dijo `cuenta`, la caché engañó a la pantalla pero no a la decisión.
Al terminar regresé el saldo de Ana a 668 y borré su copia, ese `UPDATE` rompe a propósito el cuadre entre el saldo y los movimientos, así que hay que regresarlo.

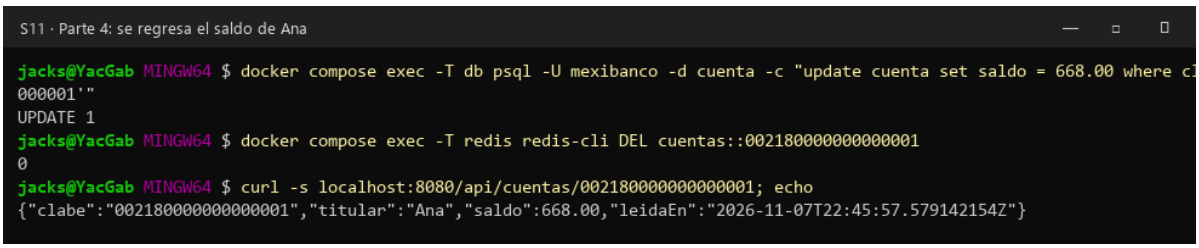


Figura 16: Se regresa el saldo real de Ana.

7 Reto: Redis se cae

7.1 Sin protección

Con la versión de la Parte 1 apagué Redis, el GET del saldo dio **500** y la transferencia también **500**, en el log de `cuenta` se ven los `RedisConnectionFailureException` y los timeouts de Lettuce, o sea que una copia que era solo para mostrar tumbó la consulta y la transferencia. Los saldos no se movieron porque la transferencia falló antes del cargo, al consultar el destino (la saga consulta por el mismo GET cacheado).

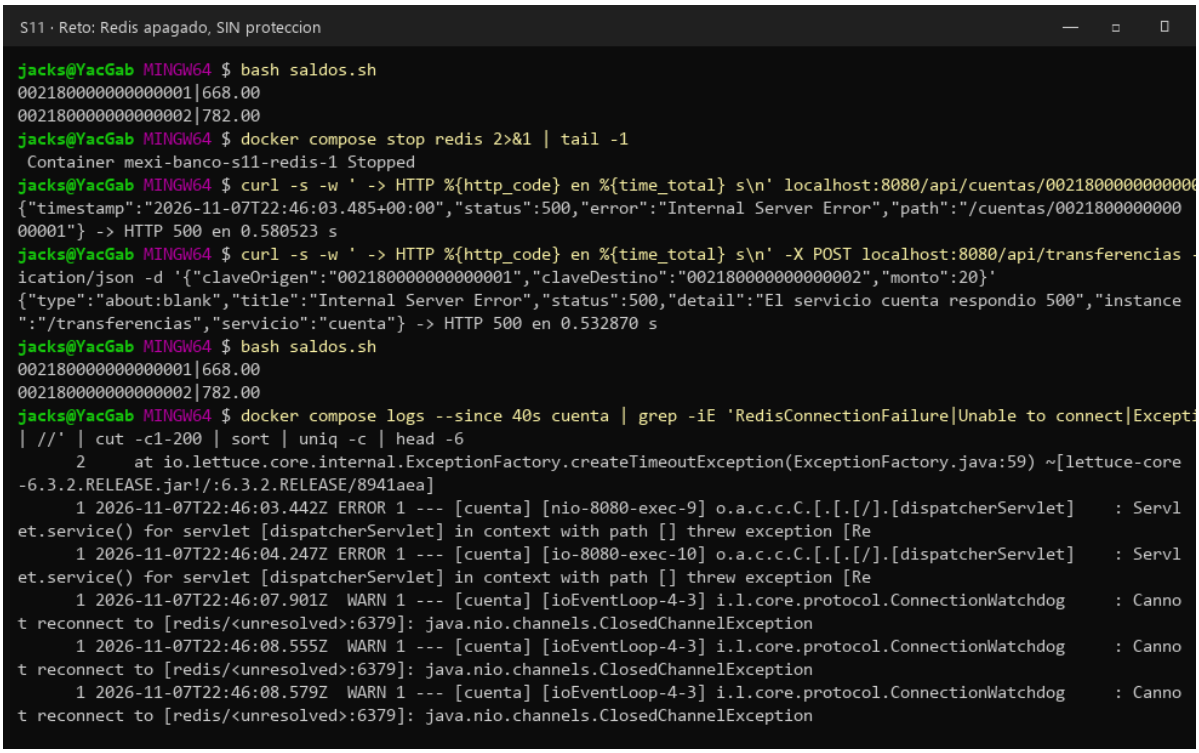


Figura 17: Redis apagado sin protección: el GET y la transferencia dan 500.

7.2 Dónde atrapar el error

La primera idea es un `CacheErrorHandler`, pero con `transactionAware()` el `DEL` de `@CacheEvict` no corre dentro del aspecto de caché sino después del commit, y ahí el `CacheErrorHandler` ya no lo atrapa, si ese `DEL` truena, `cuenta` contesta 500 a un cargo que **ya se confirmó** en Postgres, y la saga creería que el cargo no pasó. Por eso lo hice en el `RedisCacheWriter`, que es la pieza que de verdad habla con Redis: `CacheWriterTolerante` envuelve al writer de verdad, y se lo pasé al builder con `.cacheWriter(...)`, si Redis falla, el `GET` regresa `null` (un miss y se lee Postgres), y el `SET` y el `DEL` no hacen nada, todo con una línea en el log.

7.3 La primera versión creó dinero

Mi primera versión solo atrapaba la excepción. Con Redis apagado el GET ya dio 200, pero tardó 4.8 segundos, y la transferencia de 20 dio **502** en 6.8 segundos, y al revisar los saldos

Ana seguía igual pero Beto tenía **20 de más**.

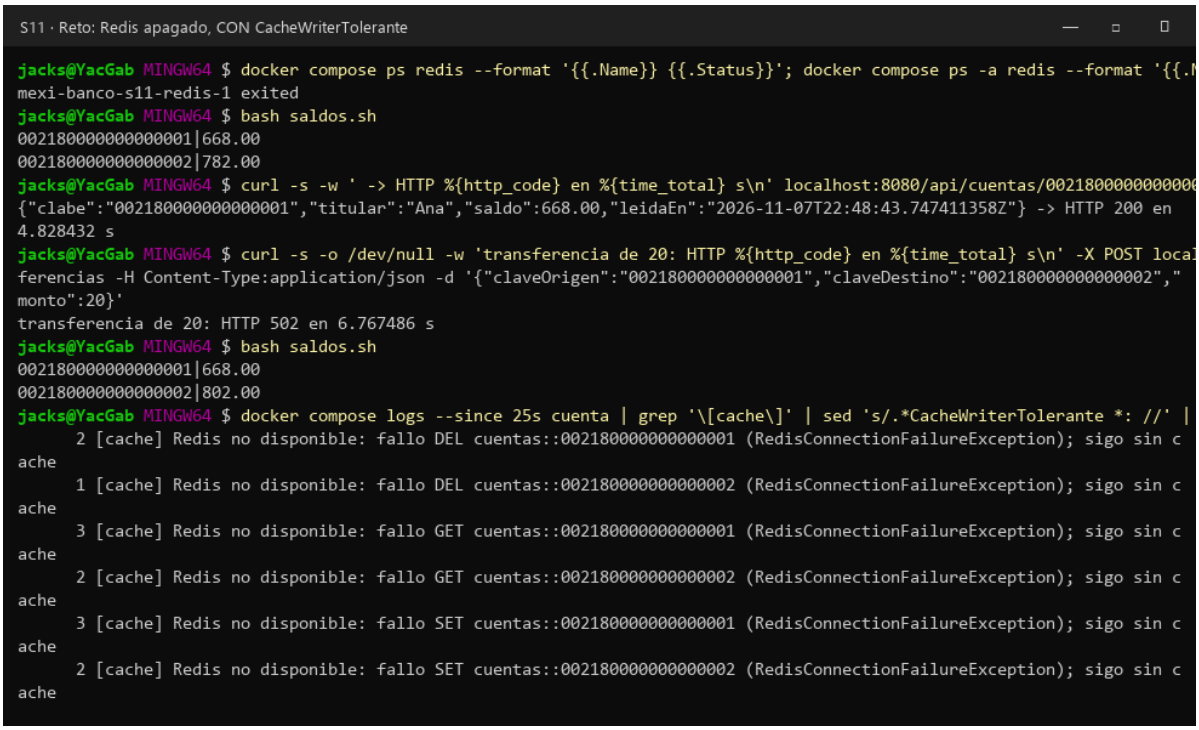


Figura 18: Primera versión: el GET da 200 pero tarda, la transferencia da 502 y Beto queda con 20 de más.

Lo que pasó es que cada intento contra un Redis apagado tardaba segundos en fallar, porque el nombre `redis` ya no resuelve y Lettuce tarda en rendirse, y ese tiempo se sumaba al abono, el abono a Beto sí se aplicó en Postgres, pero tardó más de los 2 segundos que la saga espera a `cuenta`, la saga lo dio por fallido (`Read timed out`) y compensó, le devolvió el cargo a Ana, y la transferencia 28 quedó `COMPENSADA` con el dinero en las dos cuentas, que es justo lo que pregunta la pista, una caché lenta hizo que un abono confirmado pareciera fallido. Corregí los 20 de Beto a mano y probé una segunda versión, bajando el `connect-timeout` de Redis a 200 ms y sin volver a intentar Redis durante 10 segundos después de una falla, pero el primer intento de cada copia seguía tardando unos 6.7 segundos y la transferencia 30 volvió a dar 502 con los mismos 20 de más.

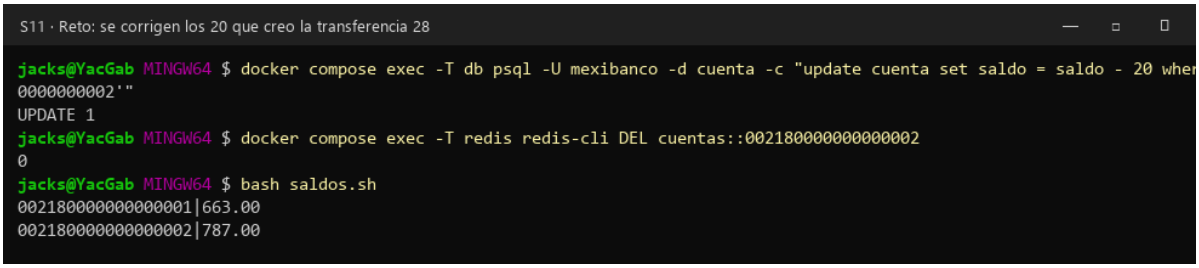


Figura 19: Corrección de los 20 de más de Beto después de la transferencia 28.

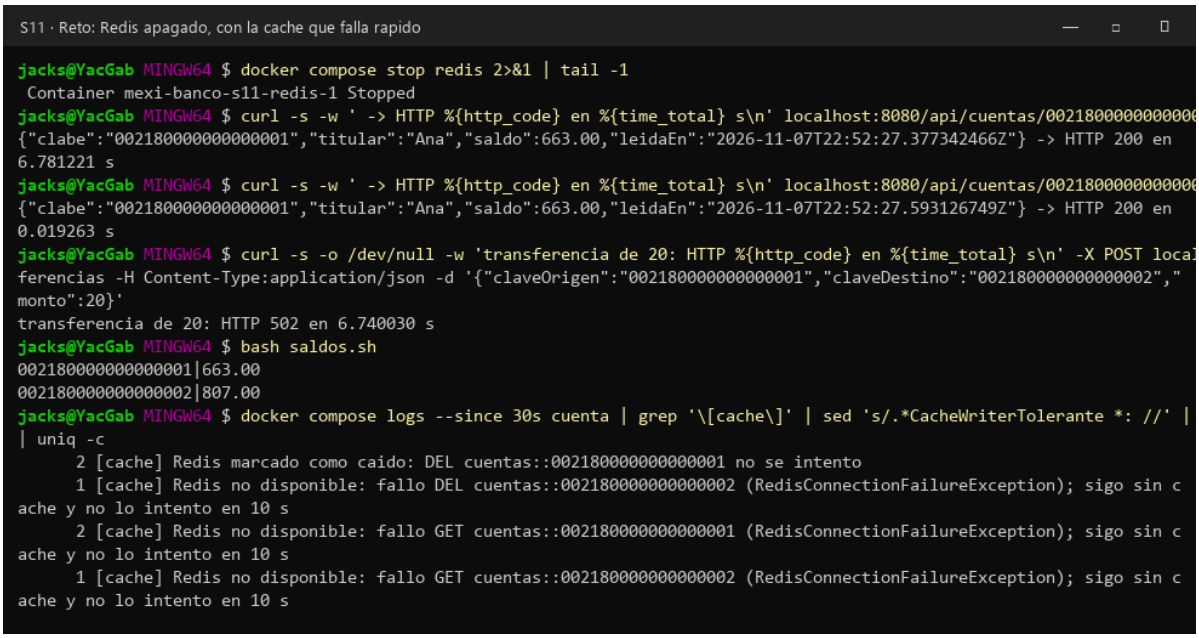


Figura 20: Segunda versión: el primer intento todavía tarda 6.7 s y la transferencia vuelve a dar 502.

7.4 La versión final

Entonces le puse a cada operación de caché un **límite duro de 300 ms**: la llamada a Redis corre en un hilo aparte y se espera con `future.get(300, MILLISECONDS)`, si no contesta en ese tiempo se da por fallida, y durante los siguientes 10 segundos ni se intenta, así una caché caída le suma como mucho 300 ms a una operación, nunca segundos.

```
private <T> T intentar(String operacion, byte[] key, Supplier<T> llamada, T siFalla) {
    if (System.currentTimeMillis() < sinRedisHasta) return siFalla; // fallo hace poco:
        ni lo intento
    Future<T> futuro = hilos.submit(llamada::get);
    try {
        return futuro.get(LIMITE_MS, TimeUnit.MILLISECONDS); // 300 ms y no mas
    } catch (TimeoutException tardo) {
        futuro.cancel(true);
        falla(operacion, key, "no␣contesto␣en␣" + LIMITE_MS + "␣ms"); // log y 10 s sin
            Redis
    } catch (ExecutionException e) {
        falla(operacion, key, e.getCause().getClass().getSimpleName());
    } ...
    return siFalla;
}
```

Corregí otra vez los 20 de Beto (la transferencia 30) y repetí la prueba, con Redis apagado el GET respondió **200** en menos de un segundo, la transferencia de 20 respondió **201** en 1.4 segundos, y los saldos cuadran, Ana pasó de 658 a 638 y Beto de 792 a 812, en el log de `cuenta` se ve qué operación falló (`GET` y `DEL` con `no contesto en 300 ms`) y cuáles ya ni se intentaron.

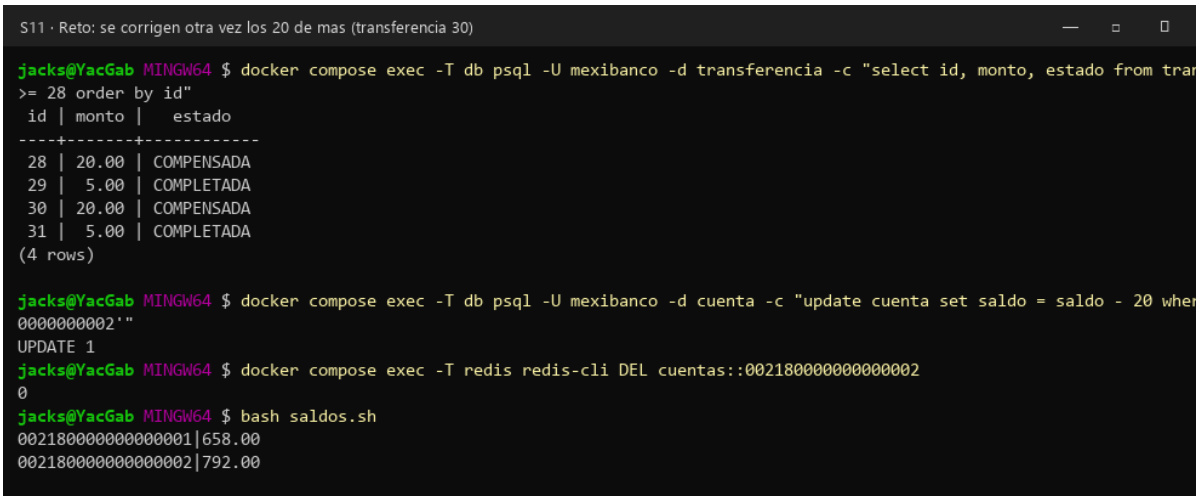


Figura 21: Se corrigen los 20 de la transferencia 30 antes de la prueba final.

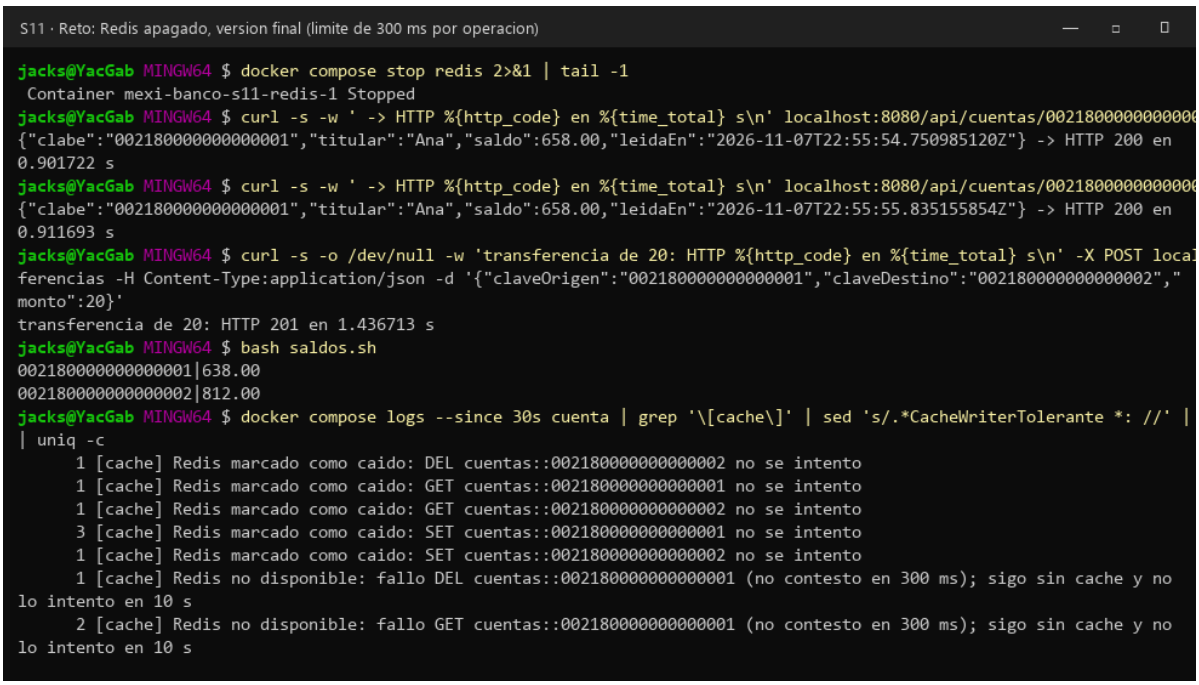


Figura 22: Versión final con Redis apagado: el GET da 200, la transferencia 201 y los saldos cuadran; el log dice qué operación de caché falló.

Al encender Redis todo siguió, los GET volvieron a llenar la copia en Redis, una transferencia de 5 dio 201, la copia de Ana se borró (`EXISTS` en 0) y los saldos quedaron en 633 y 817.

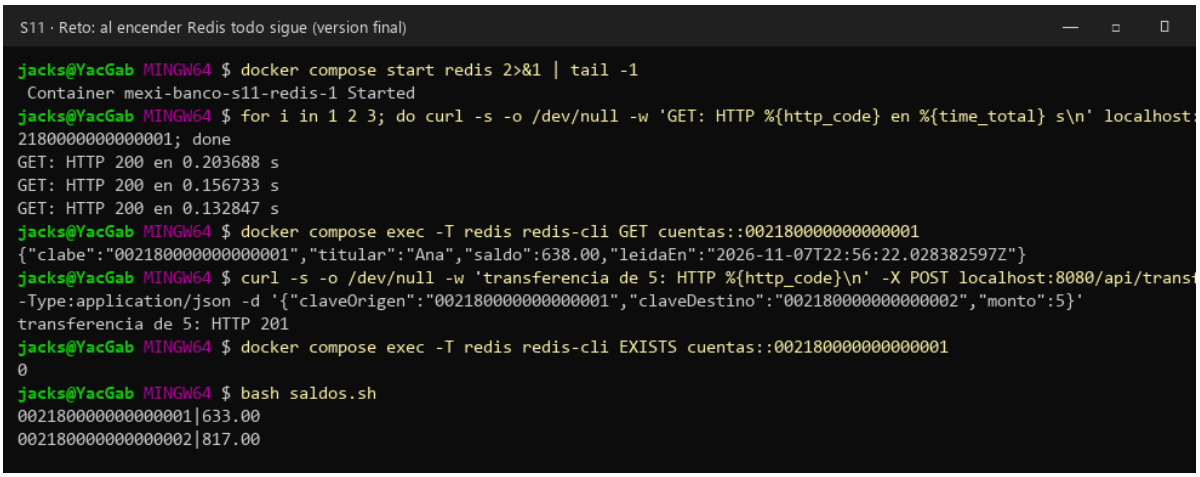


Figura 23: Redis otra vez arriba: la caché vuelve a llenarse y a borrarse, y todo sigue.

El riesgo que acepto es que, si Redis no se cayó sino que se desconectó un momento, un `DEL` perdido deja una copia vieja hasta que caduque su TTL, como en la Parte 4, para mostrar el saldo se vale, porque ninguna decisión sale de la caché. Los dos `UPDATE` que hice para quitarle a Beto los 20 de más dejan su saldo bien pero su estado de cuenta tiene el abono que la saga dio por fallido, en un banco de verdad eso sería un ajuste con su propio movimiento, y lo que de verdad lo evita es que el abono no tarde por culpa de la caché.

8 Preguntas de cierre

1. ¿Por qué MongoDB para los avisos y no para el saldo?
Porque cada aviso tiene una forma distinta según el canal, el SMS lleva teléfono, el correo lleva dirección y asunto y el push un token, y en Mongo cada uno guarda solo sus campos sin tablas con columnas vacías ni `ALTER TABLE`, además se escriben una vez, nunca se actualizan y se leen enteros por CLABE, que es justo lo que hace `ultimos()` con su índice. El saldo es lo contrario, es un solo número que cambia todo el tiempo y que tiene que cuadrar con los movimientos, necesita transacciones, el `@Version` para que dos cargos al mismo tiempo no se pisen, y que nunca se pierda ni se contradiga, eso lo da Postgres y no gano nada moviéndolo.
2. ¿Qué hit ratio medí y qué compré con la caché?
En A medí 99 % (99 hits y 1 miss) y en B 84 % (101 hits y 19 misses), baja porque cada transferencia borra la copia de Ana y la de Beto y la siguiente lectura de cada uno falla. En latencia casi no gané, la mediana pasó de 16 a 18 ms sin caché a 14 a 16 ms con caché, pero las lecturas a Postgres pasaron de 100 de 100 a 0 o 1, entonces lo que compré fue **capacidad** más que velocidad, Postgres queda libre para lo que solo él puede hacer, los cargos y los abonos.
3. ¿Qué pasaría si la autorización del cargo leyera el saldo de la caché?
En la Parte 4 la caché decía 668 cuando Postgres ya tenía 5, si el cargo se autorizara con la caché, la transferencia de 100 hubiera pasado y Ana habría quedado en −95, gastando dinero que no tiene, y no solo con un `UPDATE` raro, durante los 30 segundos del TTL o entre

el commit y el `DEL`, dos cargos seguidos podrían autorizarse con el mismo saldo viejo. Por eso la caché solo se usa para mostrar y la autorización siempre lee Postgres.

4. ¿Por qué `DEL` y no `SET` con el saldo nuevo? ¿Y por qué después del commit? Con `SET` habría dos escrituras compitiendo, si dos movimientos de la misma cuenta terminan casi juntos, el `SET` del más viejo puede llegar después y dejar en Redis un saldo que ya no es, y se quedaría ahí hasta el TTL, con `DEL` no importa el orden, la copia desaparece y la siguiente lectura la arma con lo que diga Postgres en ese momento. Y va después del commit porque si borrara antes, en el rato entre el `DEL` y el commit otra consulta podría leer el saldo viejo de Postgres, que todavía no tiene el cambio, y volver a dejarlo en la caché por 30 segundos, después del commit, cualquier lectura ya trae el saldo nuevo.

9 Conclusiones

Lo que más me quedó es que lo importante no es elegir SQL o NoSQL, sino saber qué dato es la verdad y cuál es una copia, el saldo se quedó en Postgres, los avisos se fueron a Mongo porque tienen forma de documento, y Redis guarda una copia que se vale que esté vieja un rato pero que nunca decide nada, la Parte 4 lo enseñó muy bien, la pantalla mostró 668 durante 35 segundos pero la transferencia igual dio 422. Con mis números la caché casi no hizo más rápida la consulta, lo que hizo fue quitarle a Postgres 99 de cada 100 lecturas. Y el reto me enseñó algo que no esperaba, no basta con que la caché no tire errores, también tiene que fallar rápido, porque una caché lenta hizo que un abono que sí se hizo pareciera fallido y la saga creó dinero, una pieza que era solo para mostrar terminó metida en la operación del dinero, y por eso cada operación de caché tiene ahora un límite de 300 ms.